Lab 7: Draw the State Transition Diagram

Name: Gyan Prakash Tiwari
Roll No.: 202401100200146
Branch: CSE

Project: AI Chatbot for University Helpdesk (RAG-based) |
github.com/Xxierrr/AI-Chatbot-for-University-Helpdesk

1. Aim

To model the dynamic behaviour of the **AI Chatbot for University Helpdesk** using UML state transition (state machine) diagrams, showing how key objects change state in response to events.

2. Theory

A **state transition diagram** describes the life cycle of one object or subsystem. It consists of: **states** (rounded rectangles), **transitions** (arrows), **events** that trigger them, optional **guard conditions** in square brackets `[condition]`, **actions** after a slash, an **initial state** (filled dot) and a **final state** (bull's-eye). It is most useful for objects whose behaviour depends on their history, such as a chat request or an uploaded document.

3. Objects Selected from the Project

- **Chat message** – from the moment a student sends a query until the answer and optional feedback are recorded (JWT check, retrieval, LLM generation).
- **Knowledge-base document** – from admin upload through validation, text extraction, chunking, embedding and indexing in ChromaDB until deletion.

4. State Descriptions (Chat Message)

State	Description
Idle	Chat UI waiting for user input.
Message Received	FastAPI receives the query and conversation id.
Validating JWT	Token is decoded and role (Student/Admin) checked; an invalid token returns 401 and the system returns to Idle.
Retrieving Context	Query is embedded (all-MiniLM-L6-v2) and the top-k similar chunks are fetched from ChromaDB.
Generating Answer	Retrieved chunks plus the question form the prompt sent to the LLM; the answer cites its source documents.
Unanswered Logged	No relevant chunk found, so the question is stored for the admin's 'unanswered questions' view and a fallback reply is returned.
Answer Delivered	Response is stored in conversation history and shown with Markdown rendering.
Feedback Recorded	Thumbs up/down saved through POST /api/feedback, then the system returns to Idle.

5. Diagram A: State Machine of a Chat Message

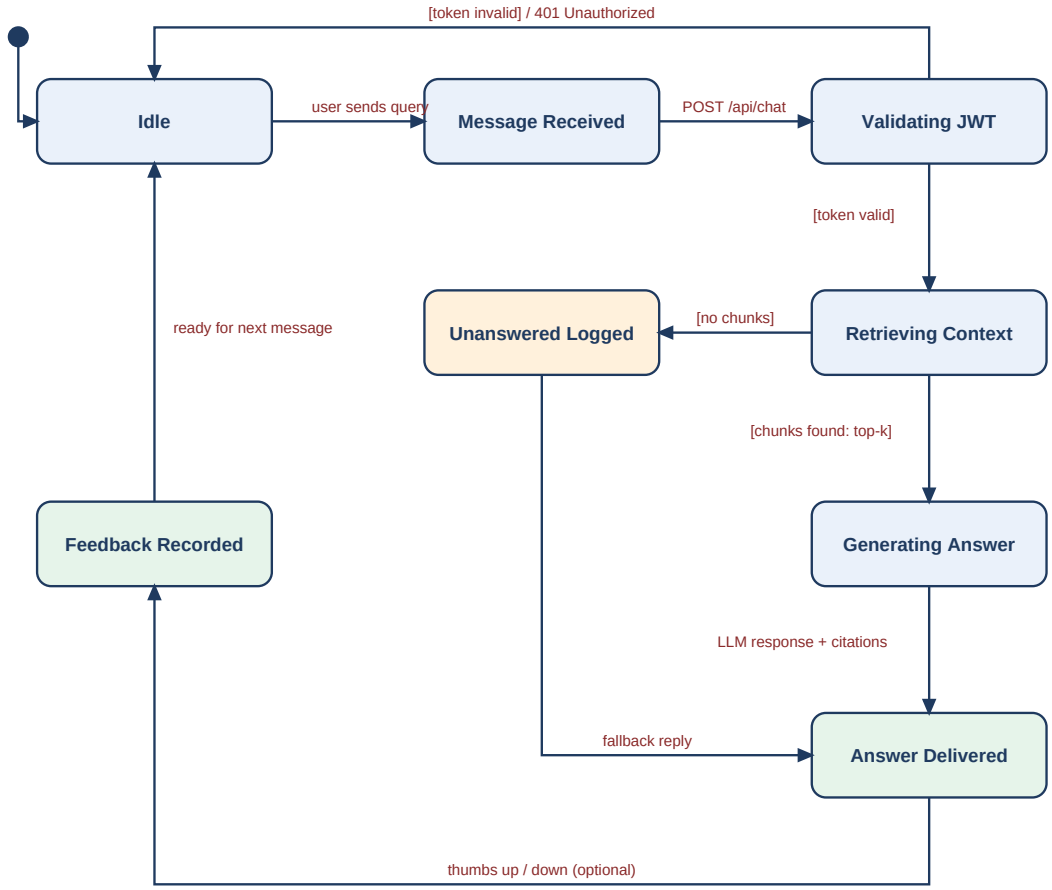

Figure 7.1 – Chat message life cycle (POST /api/chat)

6. Diagram B: State Machine of a Knowledge-Base Document

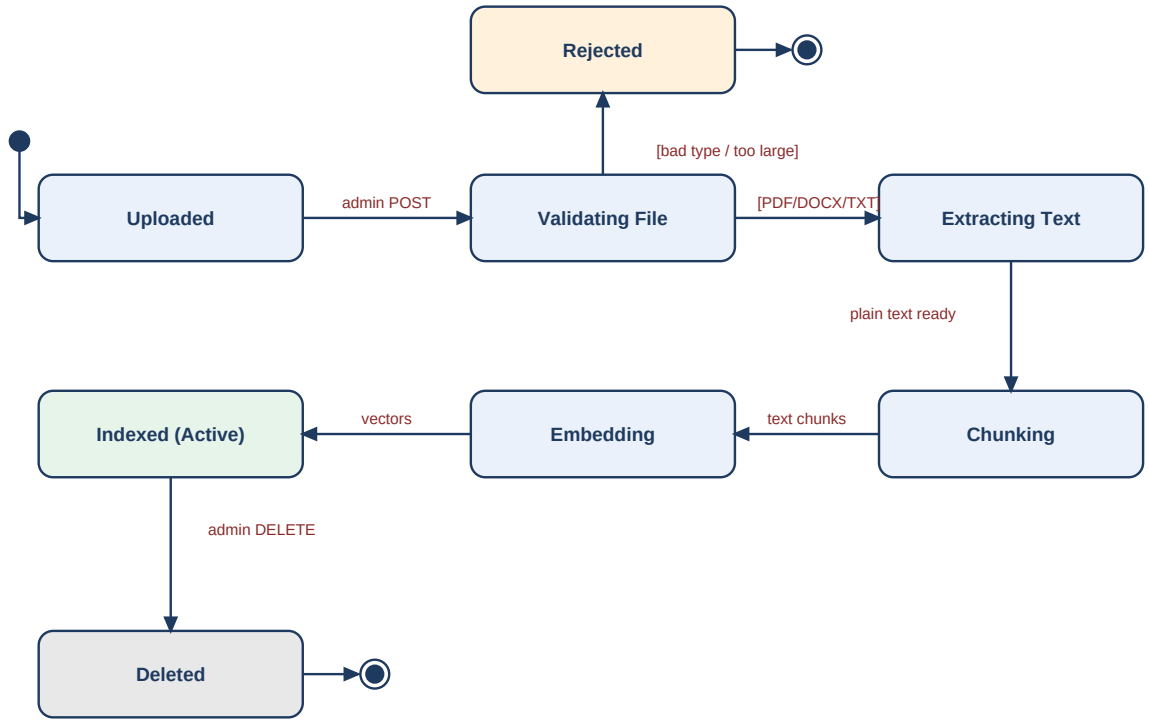

Figure 7.2 – Document life cycle in the admin knowledge-management module

Only valid file types (PDF, DOCX, TXT) pass validation. After text extraction the document is split into chunks, each chunk is converted into a vector and stored in ChromaDB, making it immediately searchable (*Indexed*). When an admin deletes the document, its vectors and record are removed (*Deleted*), so the chatbot can no longer cite it.

7. Conclusion

The state diagrams make the guard conditions (token validity, relevance of retrieved context, file type) explicit and show the error and fallback paths. They can be used directly to derive test cases such as: invalid token, empty retrieval and unsupported file upload.